The GlycoDiveR Workflow

Contents

- Installing and loading GlycoDiveR . . . 2
- Importing search engine outputs . . . 2
  - Generating the annotation file . . . 2
  - Importing the search engine output . . . 3
  - Importing comparison results . . . 3
- The GlycoDiveR data format . . . 4
- Subsets of peptides or proteins of interest . . . 5
- GlycoDiveR general plots . . . 5
  - PlotQuantificationQC . . . 5
  - PlotPSMCount . . . 6
  - PlotGlycoPSMCount . . . 7
  - PlotGlycopeptideCount . . . 8
  - PlotCompletenessMatrix . . . 8
  - PlotPSMsVsTime . . . 9
  - PlotGlycositeIdsPerRun . . . 10
  - PlotUpSet . . . 11
  - PlotPTMQuantification . . . 11
  - PlotSiteQuantification . . . 12
  - PlotGlycanCompositionPie . . . 13
  - PlotGlycanCompositionBar . . . 13
  - PlotSiteGlycanCompositionBar . . . 15
  - PlotGlycansPerSite . . . 16
  - PlotGlycoProteinGlycanNetwork . . . 17
  - PlotGlycositesVsGlycans . . . 18
  - PlotSiteGlycanComposition . . . 19
  - PlotGlycanSubcellularLocation . . . 19
- GlycoDiveR comparison plots . . . 20
  - PlotSiteGlycanComposition . . . 20
- GlycoDiveR computation functions . . . 21

ComputeMissingess 21
ComputeNumberOfGlycoforms 21
Saving plots 22

This vignette provides the basics of using GlycoDiver. It will showcase the defaults for numerous graphs. For further reading please refer to the Github and for insights on specific functions type ? followed by the function name (?ImportMSFragger) in R after loading GlycoDiveR.

Installing and loading GlycoDiveR

You can install and load the development version of GlycoDiveR from GitHub with:

```
if (!requireNamespace("GlycoDiveR", quietly = TRUE)) {
  devtools::install_github("riley-research/GlycoDiveR", build_vignettes = TRUE)
}
```

```
library(GlycoDiveR)
```

To follow this vignette, you can use the raw data uploaded on GlycoDiver's Github, but is also pre-loaded, so you don't have to follow the first few steps. If you want to use the pre-loaded data run the following code:

```
data("exampleData")
```

Importing search engine outputs

GlycoDiveR can directly import output from MSFragger (including N-glycopeptide searches and O-glycopeptide searches with or without OPair), Byonic, and pGlyco. It is also directly compatible with data processed using MSstats or Perseus (see the GitHub for details).

It is important that GlycoDiveR interfaces with untouched search engine outputs. Any manual changes might break the importer.

Every importer needs the following three things to import data:

- An annotation file
- The untouched search engine output
- The fasta file used for the search, this means including contaminants and decoys for MSFragger searches

Generating the annotation file

The annotation file is used to specify which runs belong to which condition, assign biological replicates, and assign technical replicates.

The annotation file template is automatically generated by running the following code.

```
GetAnnotationTemplate(path = "C:/pathToFolder", tool = "MSFragger")
```

It will find all `psm.tsv` files in the listed folder AND all the sub folders. Additionally, it will identify all `combined_peptide.tsv` files if FragPipe computed normalized intensities are used.

Important note: when importing MSFragger output, it is important that you have correctly supplied Experiment names in, for example, FragPipe. In FragPipe this is done in the Workflow tab. It is always a safe bet to have a unique Experiment name for each sample. If you don't, there might be issues.

An annotation.csv file will be generated. Fill in the annotation file as shown below. Make sure to use unique names in the Alias column.

```
#>                Run       Alias Condition BioReplicate TechReplicate
#> 1   Run_Treated_1   Treated_1   Treated            1             1
#> 2   Run_Treated_2   Treated_2   Treated            2             1
#> 3 Run_Untreated_1 Untreated_1 Untreated            1             1
#> 4 Run_Untreated_2 Untreated_2 Untreated            2             1
```

Importing the search engine output

Each importer has numerous arguments that control for the desired cutoffs (e.g. peptide and glycan score cutoffs, and localization confidence), normalization options, and more. Please take the time to read the docs of the importer you are using.

```
?ImportMSFragger

exampleData <- ImportMSFragger(path = "C:/pathToFolder",
                       annotation = "C:/pathToFolder/annotation.csv",
                       fastaPath = "C:/pathToFolder/database-decoys-contam.fasta.fas",
                       normalization = "median",
                       peptideScoreCutoff = 0,
                       glycanScoreCutoff = 0.01)
```

Importing comparison results

GlycoDiveR can compute log2 fold-changes, p values, and FDR values; however, for more detailed control please use Perseus or MSstats, and use GlycoDiveR's importers to import those results (see the Github wiki how to do this).

Each of the options to get comparison results will return a dataframe with the results.

```
myComparison <- GetComparison(exampleData,
                              comparisons = list(c("NP0", "NPA"),
                                                 c("NP0", "NPB"),
                                                 c("NPA", "NPB")),
                              type = "glyco")

head(myComparison,5)
#> # A tibble: 5 x 8
#>   UniprotIDs Proteins ModifiedPeptide         ModificationID Label log2FC pvalue
#>   <chr>      <chr>    <chr>                   <chr>          <chr>  <dbl>  <dbl>
#> 1 P04004     VTN      N[2204.7725]ISDGFDGIPD~ N242           NP0-~  -5.77 0.0199
#> 2 P04004     VTN      N[2204.7725]ISDGFDGIPD~ N242           NP0-~  -6.96 0.0320
#> 3 P04004     VTN      N[2204.7725]ISDGFDGIPD~ N242           NPA-~  -1.19 0.0672
#> 4 P04004     VTN      N[2204.7725]GSLFAFR     N169           NP0-~  -7.19 0.147
#> 5 P04004     VTN      N[2204.7725]GSLFAFR     N169           NP0-~  -6.94 0.113
#> # i 1 more variable: adjpvalue <dbl>
```

The GlycoDiveR data format

All GlycoDiveR importers will generate the same data format to make sure all imported data is directly compatible with all plots. Each importer returns a list containing the following objects:

- A data frame with PSM level data (acces through: exampleData$PSMTable)
- A data frame where each row is one PTM site (acces through: exampleData$PTMTable)
- The unfiltered dataframe (acces through: exampleData$rawPSMTable)
- The annotation file (acces through: exampleData$annotation)
- The inputted search engine and provided cutoffs

```
data("exampleData")
head(exampleData$PSMTable, 5)
#>   ID      Run      ModifiedPeptide PSMScore     AssignedModifications
#> 1  1 NP0_1_i1  HN[1913.6770]STGCLR   17.069 2N(1913.6770),6C(57.0215)
#> 2  2 NP0_1_i1   GHVN[1913.6770]ITR   16.774             4N(1913.6770)
#> 3  3 NP0_1_i1   GHVN[1913.6770]ITR   14.393             4N(1913.6770)
#> 4  4 NP0_1_i1 N[2059.7349]NSDISSTR   11.158             1N(2059.7349)
#> 5  5 NP0_1_i1 N[2059.7349]NSDISSTR   15.645             1N(2059.7349)
#>   TotalGlycanComposition GlycanQValue IsUnique           UniprotIDs Genes
#> 1           N(4)H(5)A(1)            0    FALSE P10909,E7ETB4,H0YC35   CLU
#> 2           N(4)H(5)A(1)            0    FALSE P00734,C9JV37,E9PIT3    F2
#> 3           N(4)H(5)A(1)            0    FALSE P00734,C9JV37,E9PIT3    F2
#> 4       N(4)H(5)F(1)A(1)            0     TRUE               P01871  IGHM
#> 5       N(4)H(5)F(1)A(1)            0     TRUE               P01871  IGHM
#>   ProteinLength NumberOfNSites NumberOfOSites ProteinStart RetentionTime
#> 1           449              6             61          290      10.08027
#> 2           622              4             74          118      11.99670
#> 3           622              4             74          118      12.01440
#> 4           474              4             98           46      12.26852
#> 5           474              4             98           46      12.28951
#>              GlycanType
#> 1 Sialylated, NonGlyco
#> 2            Sialylated
#> 3            Sialylated
#> 4     Sialofucosylated
#> 5     Sialofucosylated
#>
#> 1 Secreted;Cytoplasm;Nucleus;Mitochondrion Membrane; Peripheral Membrane Protein; Cytoplasmic Side;Cy
#> 2
#> 3
#> 4
#> 5
#>                                                                       Domains
#> 1                                                                        <NA>
#> 2                                                                        <NA>
#> 3                                                                        <NA>
#> 4 Extracellular(1-450);Cytoplasmic(472-474);Transmembrane domain(451-471)
#> 5 Extracellular(1-450);Cytoplasmic(472-474);Transmembrane domain(451-471)
#>   RawIntensity Intensity Condition    Alias BioReplicate TechReplicate
#> 1     23427926  55878462       NP0 NP0_1_i1            1             1
#> 2     51751680 123434072       NP0 NP0_1_i1            1             1
#> 3     54454740 129881200       NP0 NP0_1_i1            1             1
```

```
#> 4     133659576 318794399        NPO NPO_1_i1             1             1
#> 5     133659576 318794399        NPO NPO_1_i1             1             1
```

The GlycoDiveR dataframes will have all the unfiltered rows, and will only filter for the selected cutoffs when GlycoDiveR functions are called, for example, when plotting. If desired, you can modify the dataframes as long as the formatting remains the same

Subsets of peptides or proteins of interest

Often times you want to look at a subset of proteins or peptides. Each GlycoDiveR plot has a 'whichPeptide' and a 'whichProtein' argument. These will filter for only the peptides or proteins of interest.

whichPeptide: This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.

whichProtein: These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.

This flexible format makes it easy to create and visualize focused subsets of your data.

GlycoDiveR general plots

Each GlycoDiveR plot has numerous arguments to customize various aspects of the visualization. Please take the time to look at these arguments for the plots you are making. Please refer to the wiki on Github to learn how to change the default color palette.

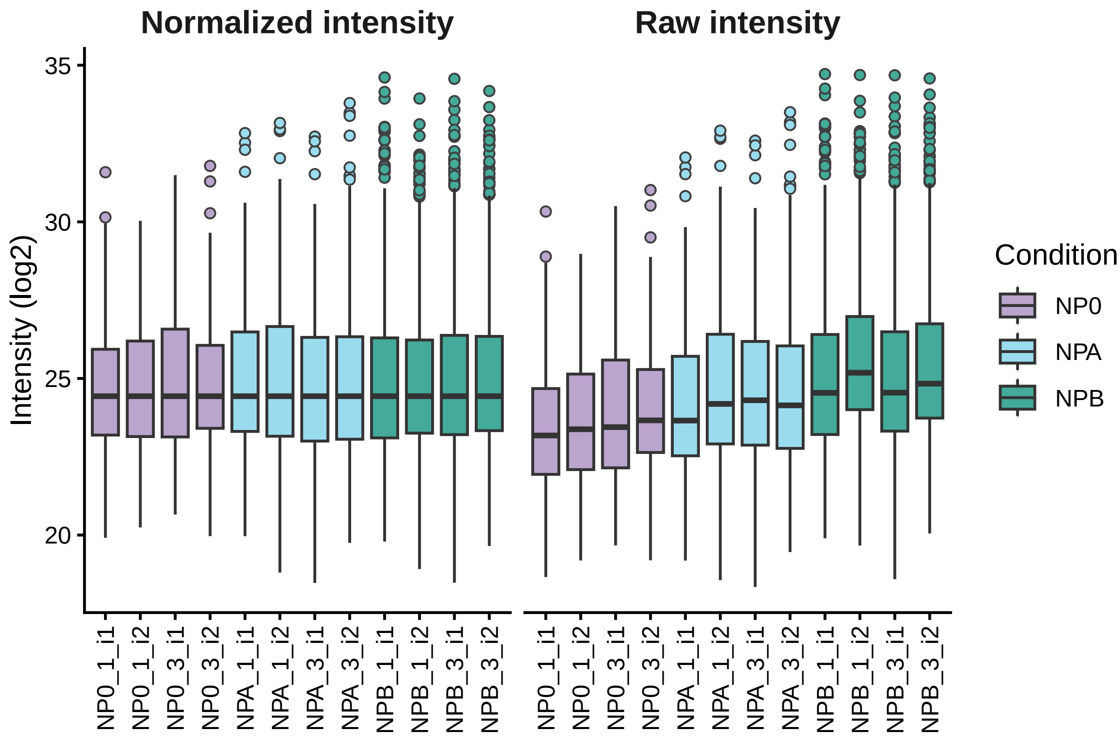
If you have haven't imported the dataset, please do so to follow along:

```
data("exampleData")
```

PlotQuantificationQC

Showcase the normalized and raw intensity values.

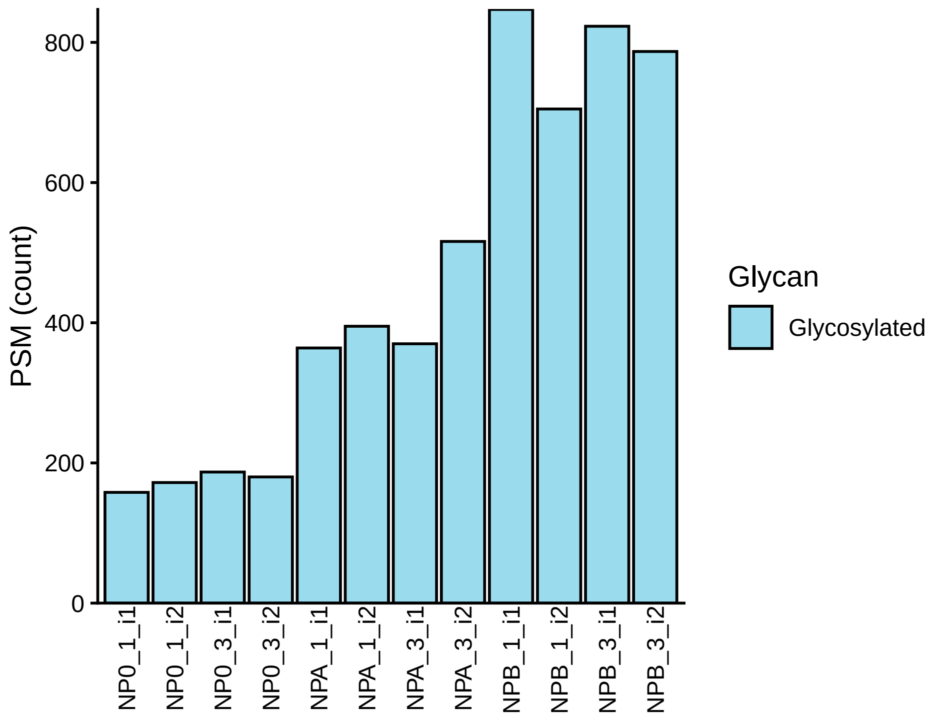
```r
PlotQuantificationQC(exampleData, whichQuantification = "both")
```



PlotPSMCount

The total number of PSMs.

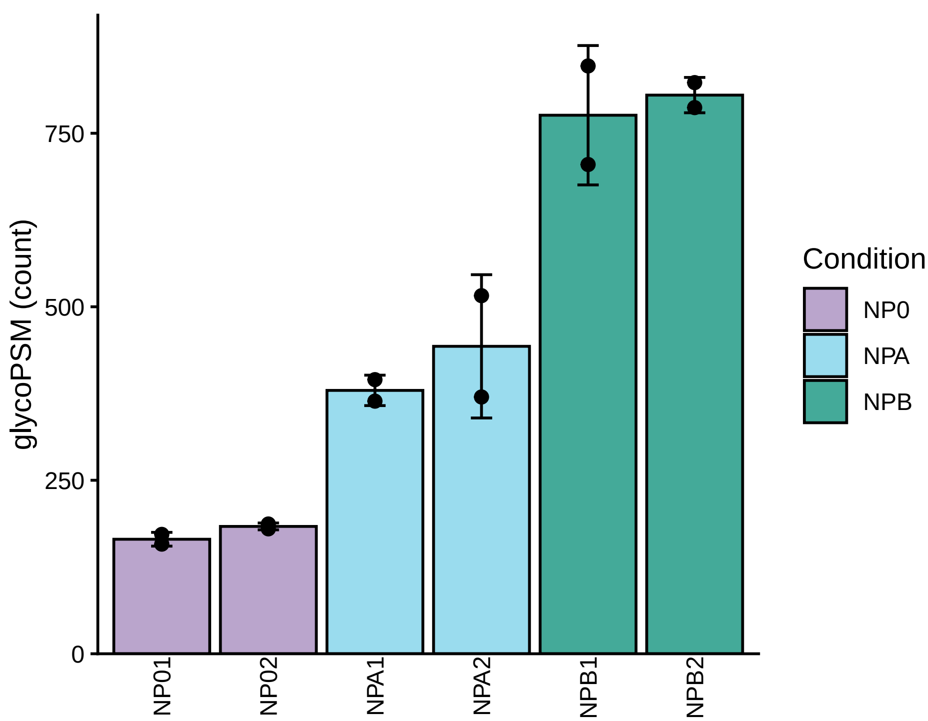
```r
PlotPSMCount(exampleData, grouping = "technicalReps")
```



PlotGlycoPSMCount

The number of glycoPSMs.

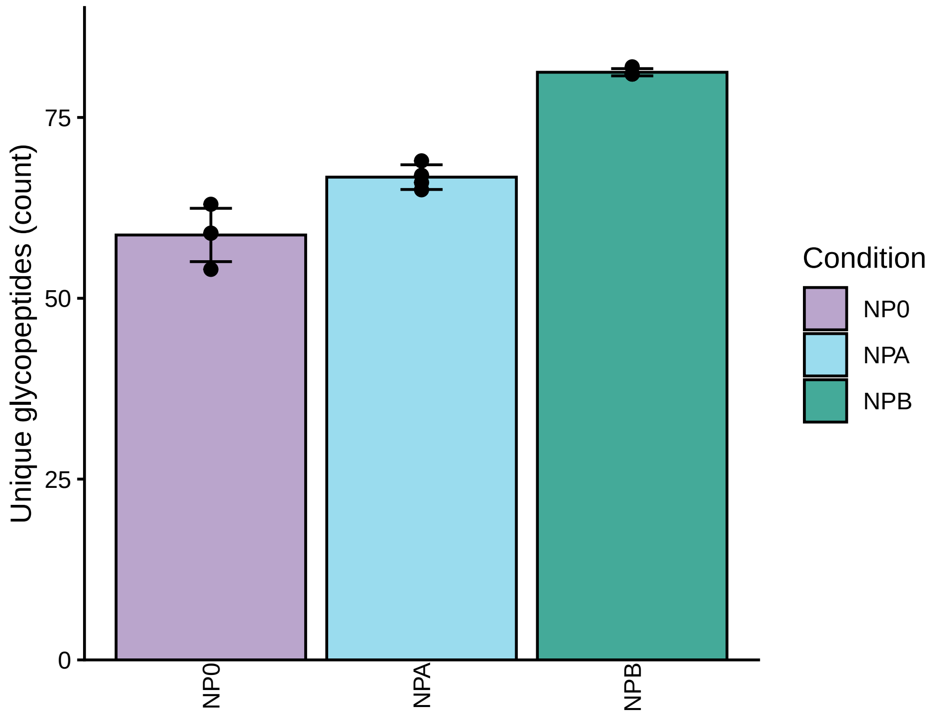
```
PlotGlycoPSMCount(exampleData, grouping = "biologicalReps")
```



PlotGlycopeptideCount

The number of unique glycopeptides.

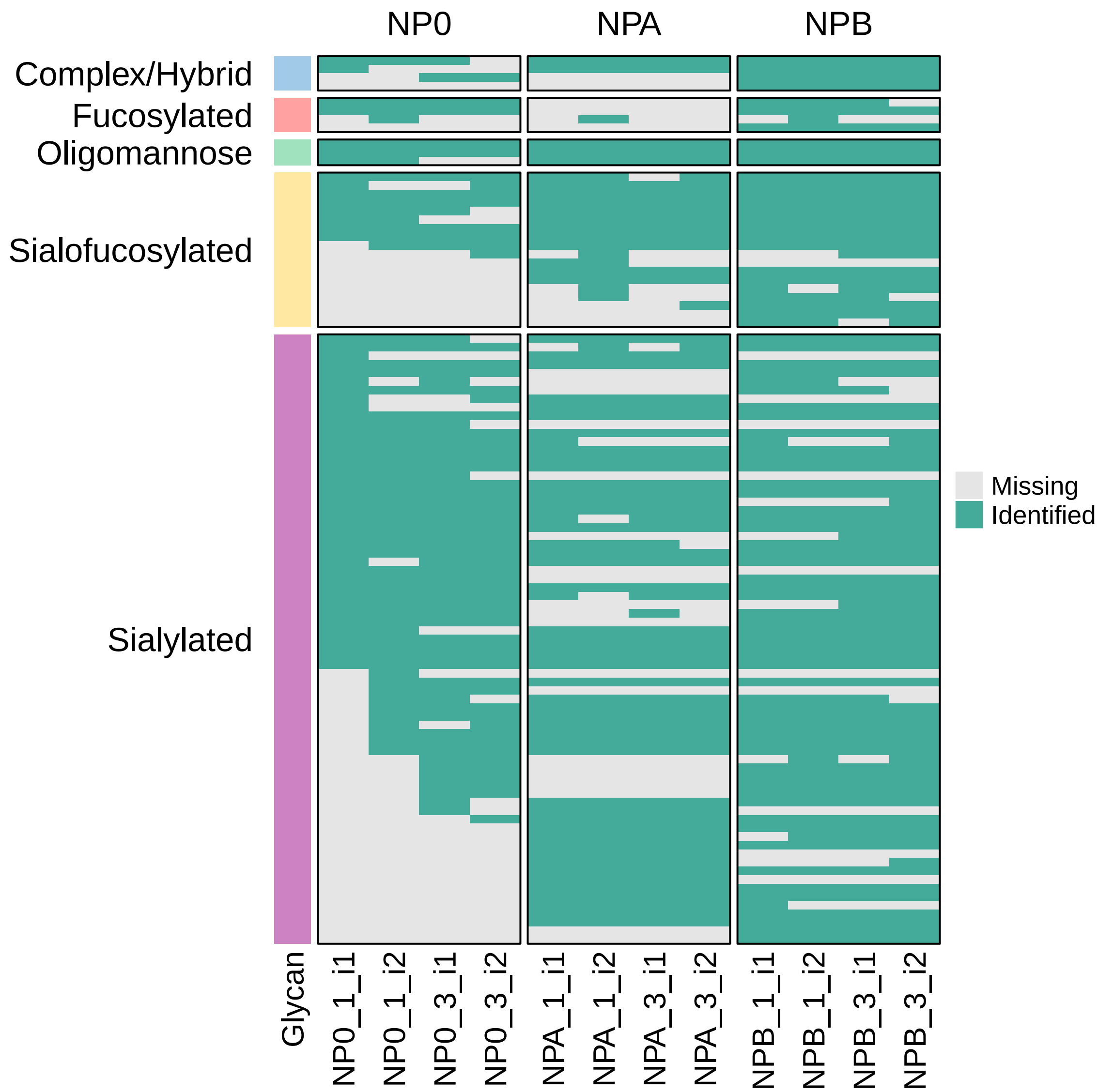
```
PlotGlycopeptideCount(exampleData, grouping = "condition")
```



PlotCompletenessMatrix

A matrix showing what (glyco)PSMs were identified in the samples. Provides insights into your data completeness.

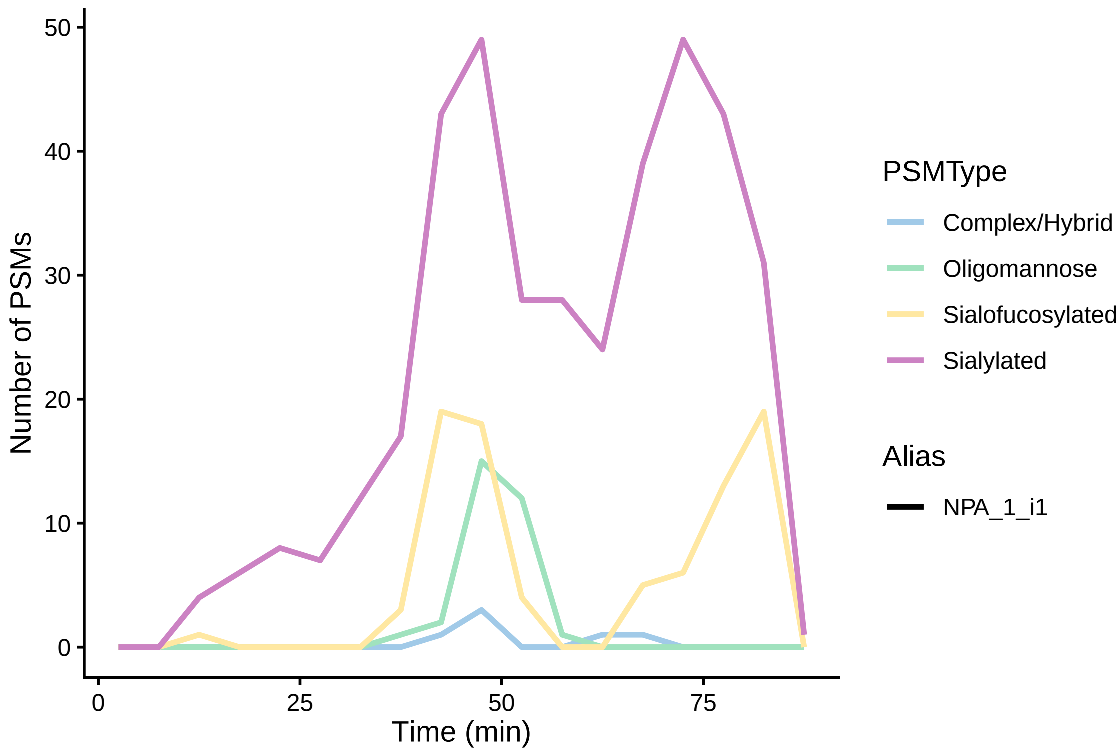
```
PlotCompletenessMatrix(exampleData, peptideType = "glyco", collapseTechReps = FALSE)
```



PlotPSMsVsTime

The retention time versus the number of identified (glyco)PSMs.

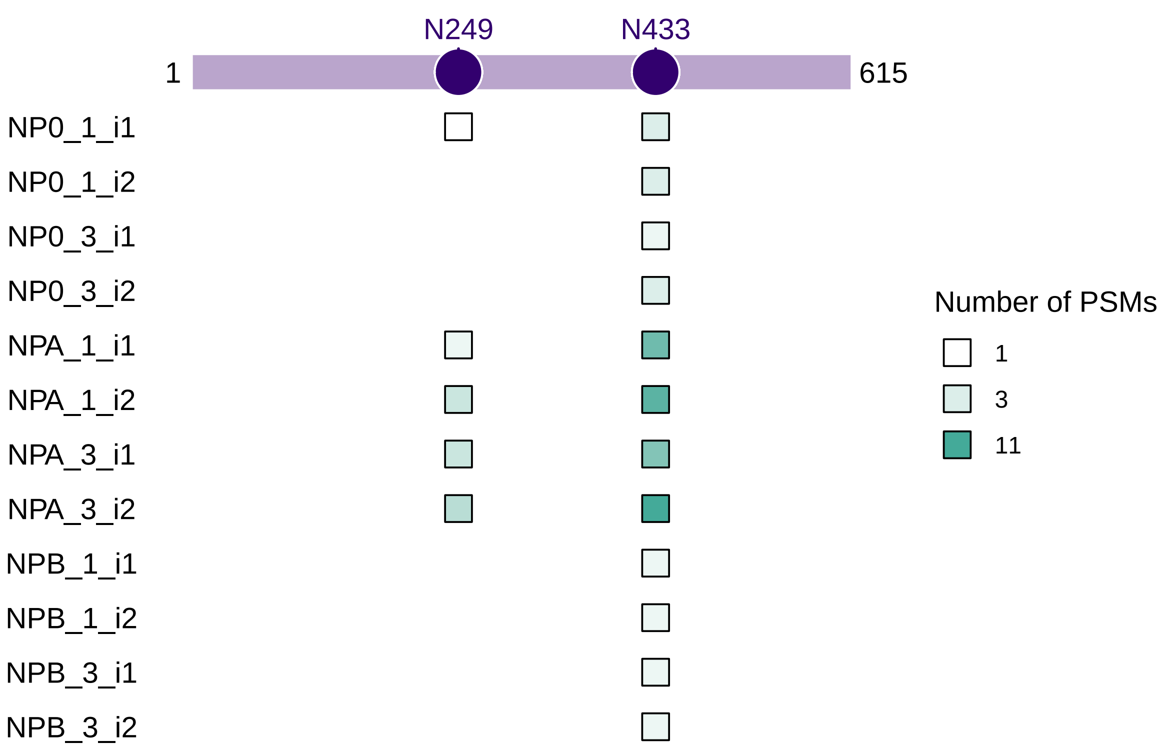
```
PlotPSMsVsTime(exampleData, type = "allGlyco", whichAlias = c("NPA_1_i1"))
```



PlotGlycositeIdsPerRun

What runs identified specific glycosylation sites.

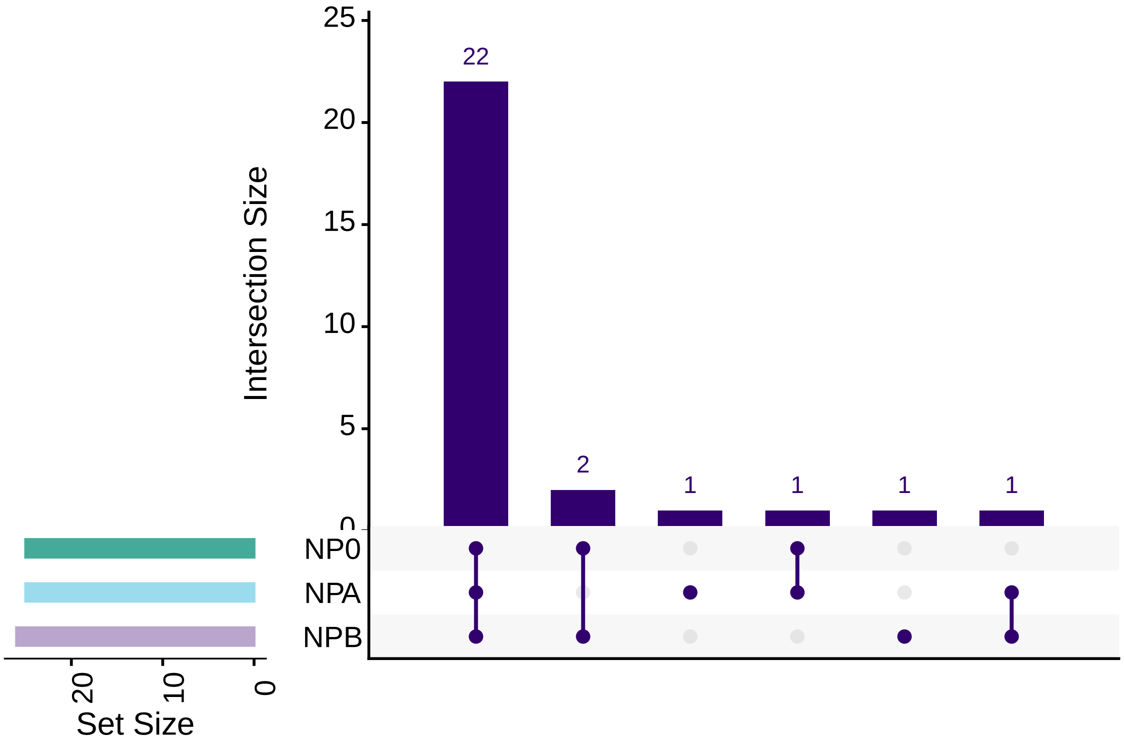
```
PlotGlycositeIdsPerRun(exampleData, whichProtein = "P00748")
```



PlotUpSet

An Upset plot to showcase overlap in glycopeptide, glycoprotein, peptide, or protein overlap.

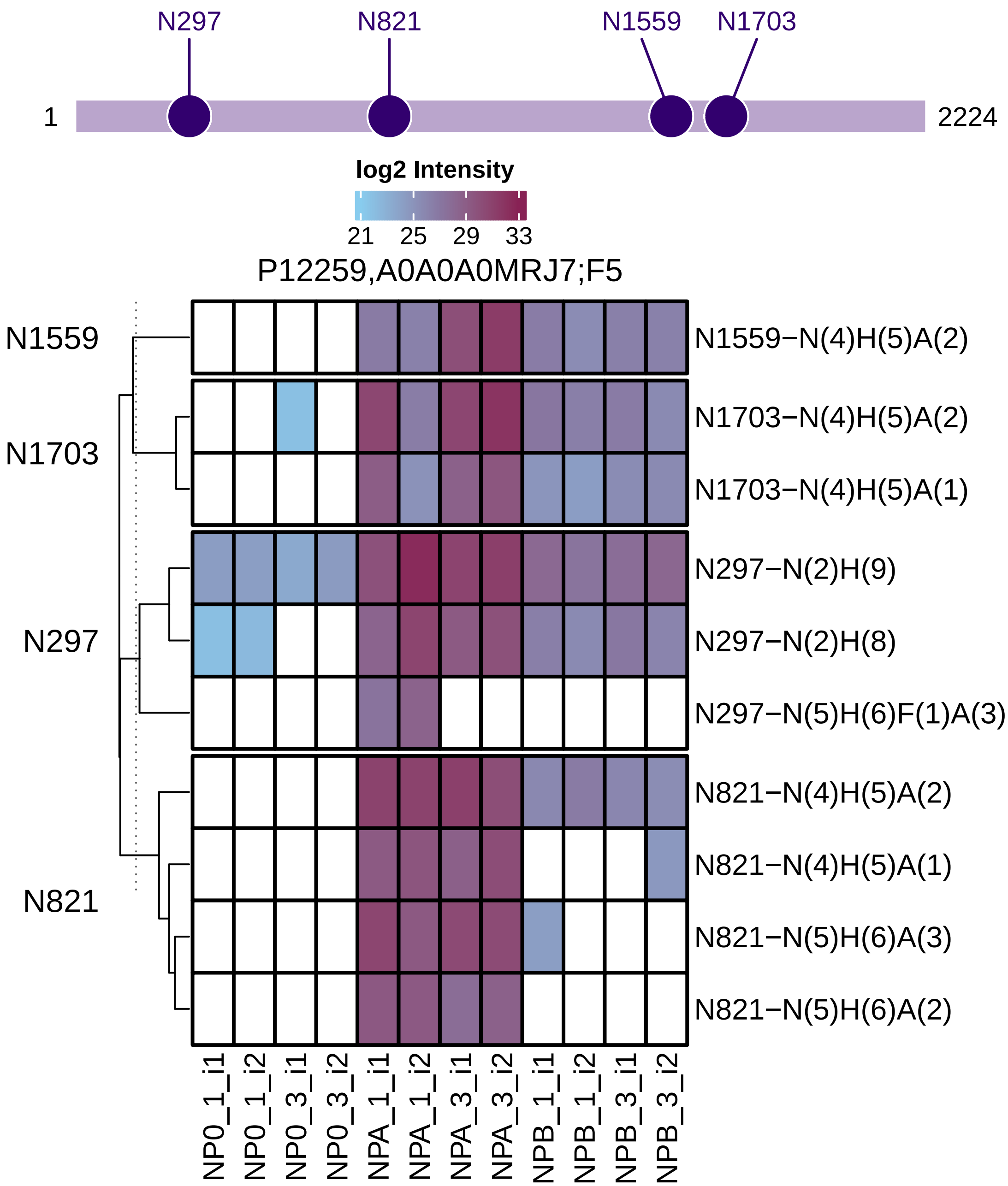
```
PlotUpSet(exampleData, type = "glyco", level = "protein")
```



PlotPTMQuantification

An heatmap showing the glycosite quantification for a single protein.

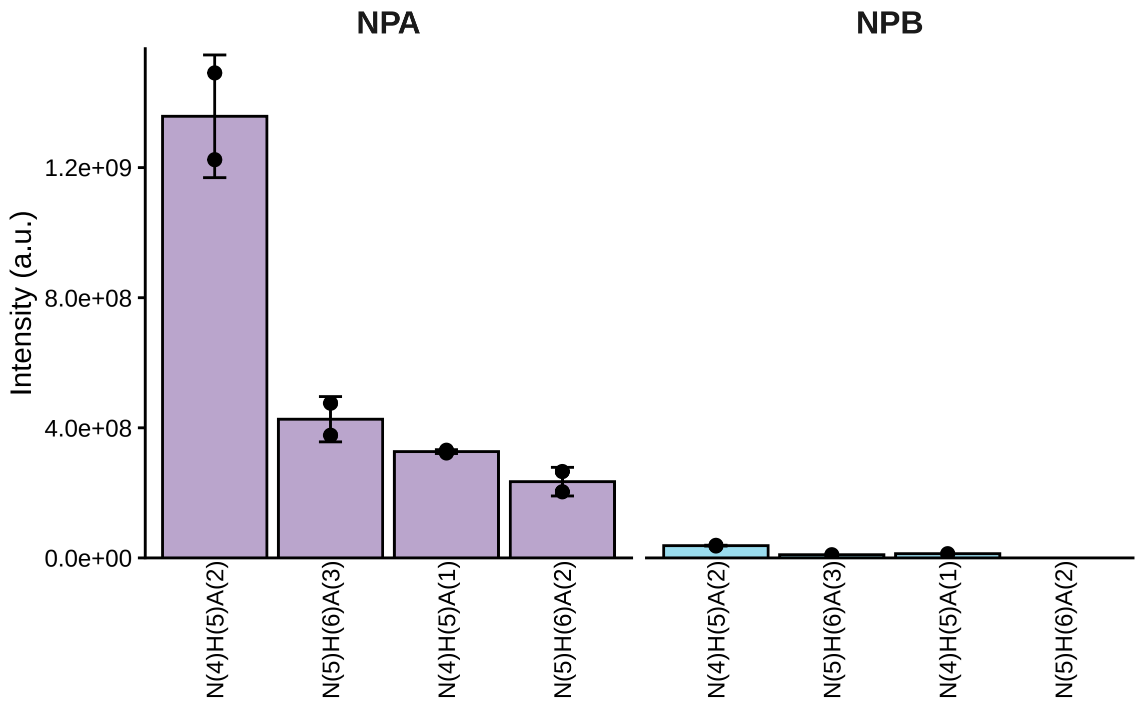
```
PlotPTMQuantification(exampleData, whichProtein = "P12259,A0A0A0MRJ7")
```



PlotSiteQuantification

Quantification of a single glycosite.

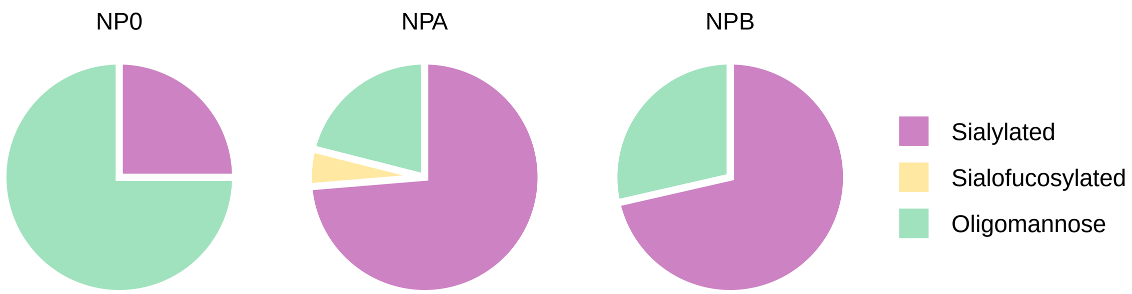
```
PlotSiteQuantification(exampleData, whichProtein = "P12259,A0A0A0MRJ7", site = "N821")
```



PlotGlycanCompositionPie

The glycan composition of a single protein or a list of protein.

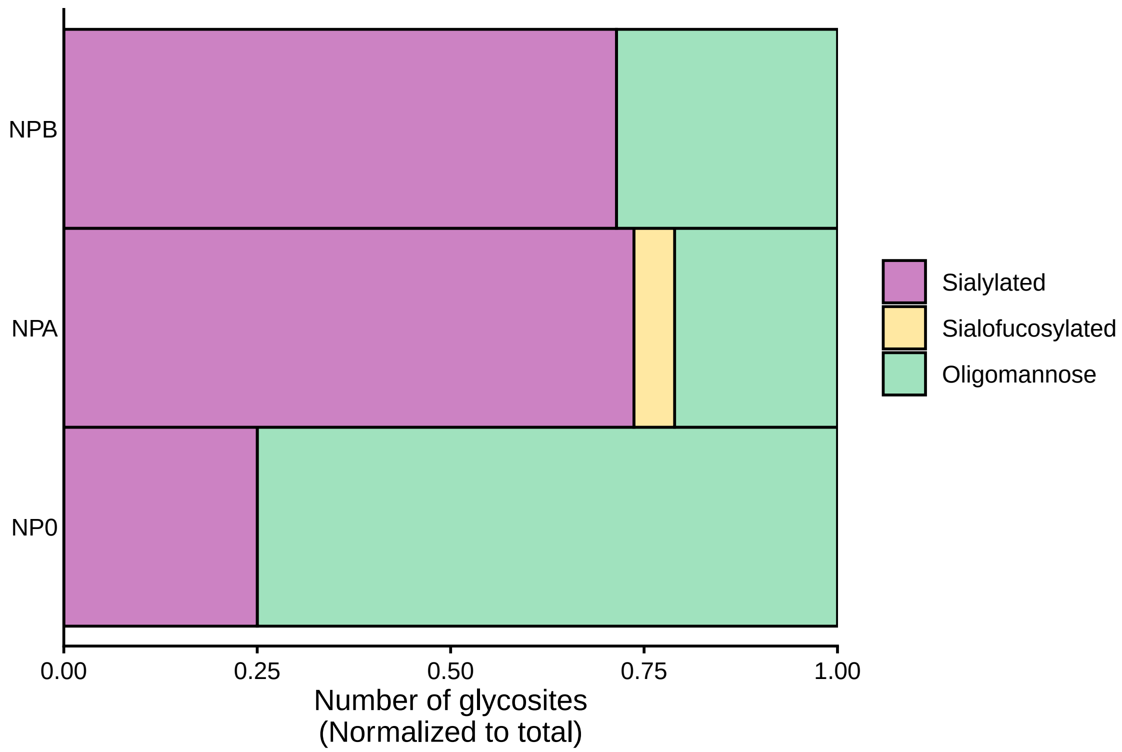
```
PlotGlycanCompositionPie(exampleData, whichProtein = "P12259,A0A0A0MRJ7")
```



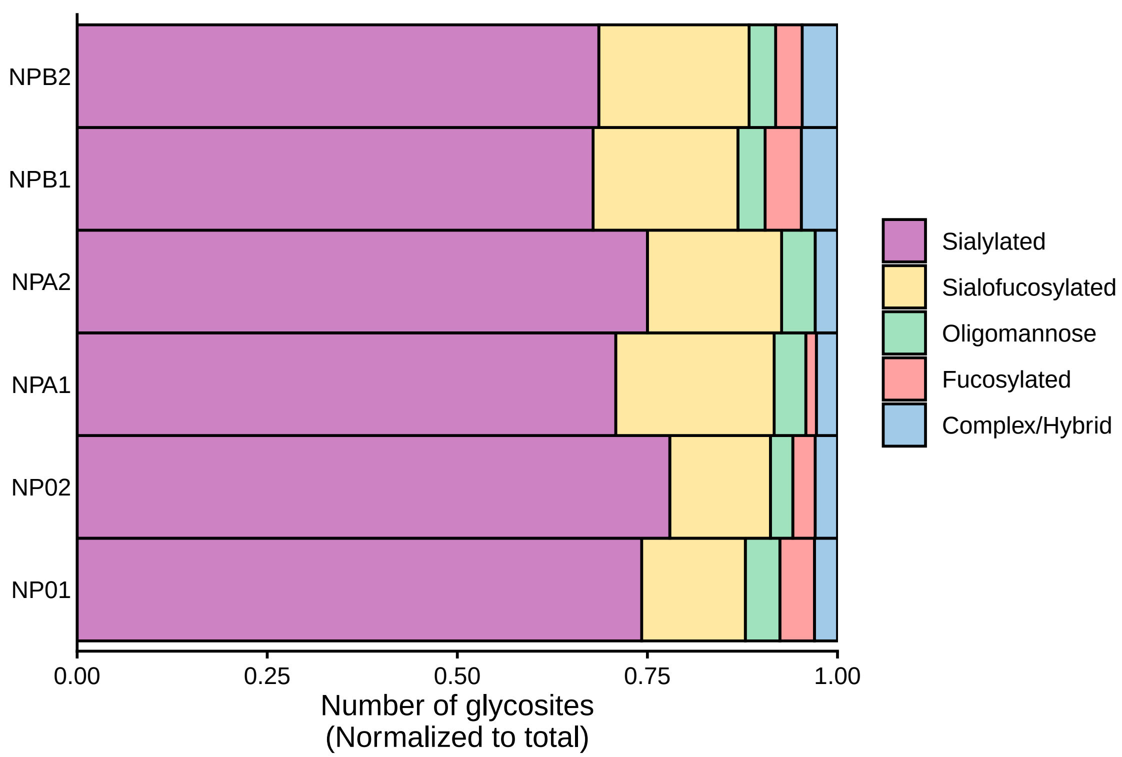
PlotGlycanCompositionBar

The glycan composition of a single protein or a list of protein.

```
PlotGlycanCompositionBar(exampleData, whichProtein = "P12259,A0A0A0MRJ7",
                         grouping = "condition")
```



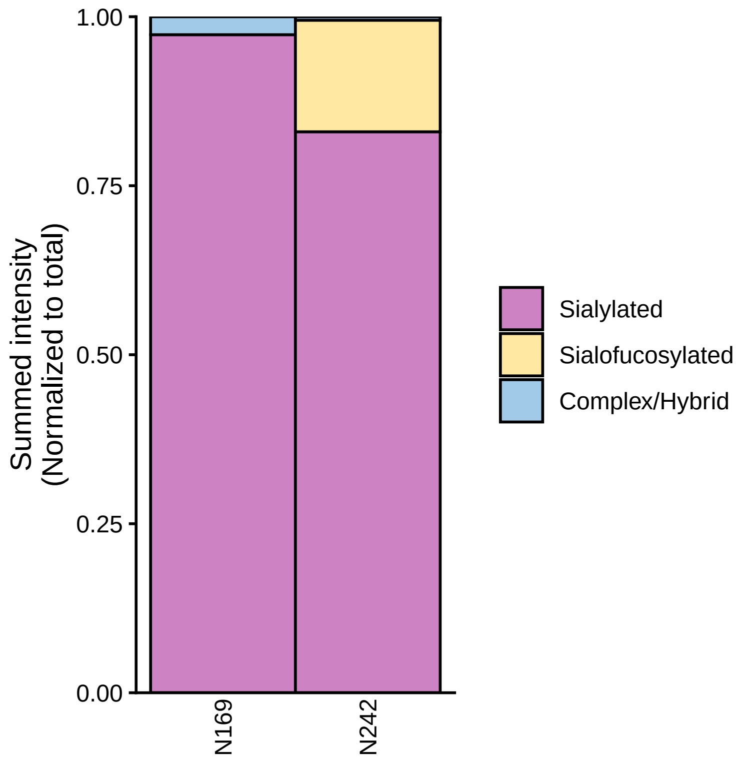

```r
PlotGlycanCompositionBar(exampleData, grouping = "biologicalReps")
```



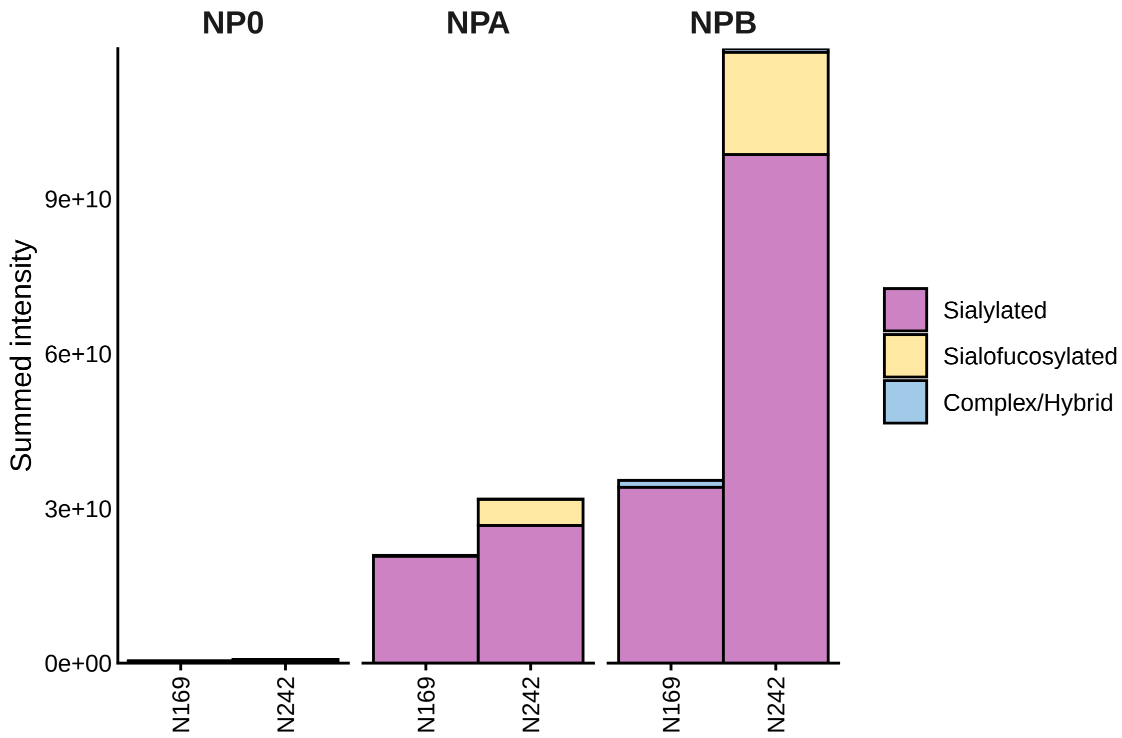
PlotSiteGlycanCompositionBar

Plot the glycan composition per protein. This visualization can be normalized to total, and grouped by condition, biological reps, technical reps, or ungrouped using all data.

```
PlotSiteGlycanCompositionBar(exampleData, grouping = "none",
                             whichProtein = "P04004", scales = "fill")
```



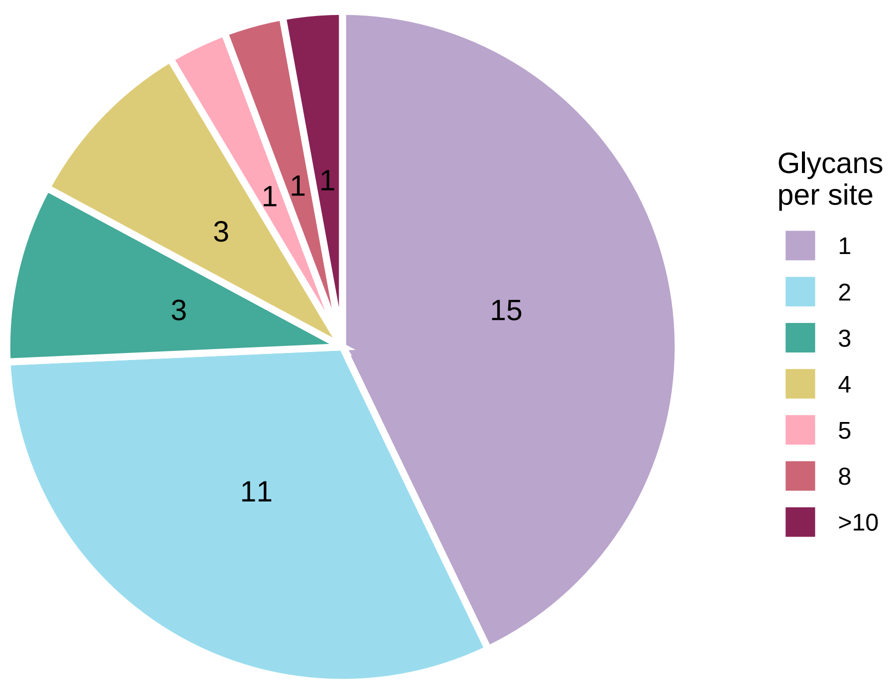

```
PlotSiteGlycanCompositionBar(exampleData, grouping = "condition",
                             whichProtein = "P04004", scales = "stack")
```



PlotGlycansPerSite

This plot shows how many glycans were detected at each site— that is, how many sites have 1 glycan, 2 glycans, and so on.

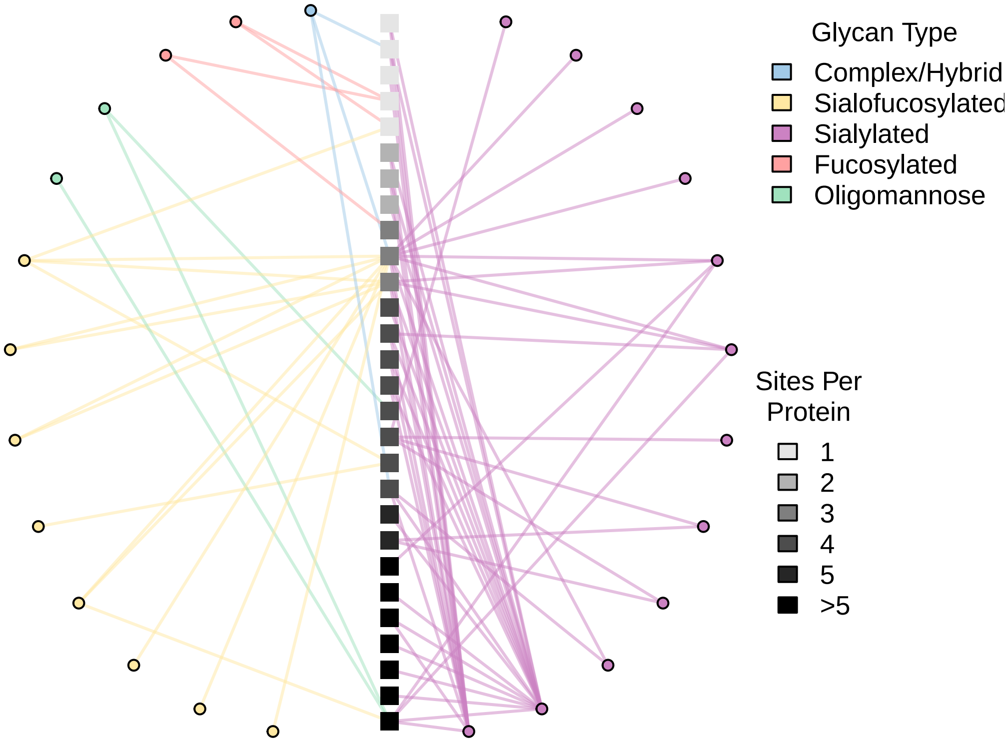
```
PlotGlycansPerSite(exampleData)
```



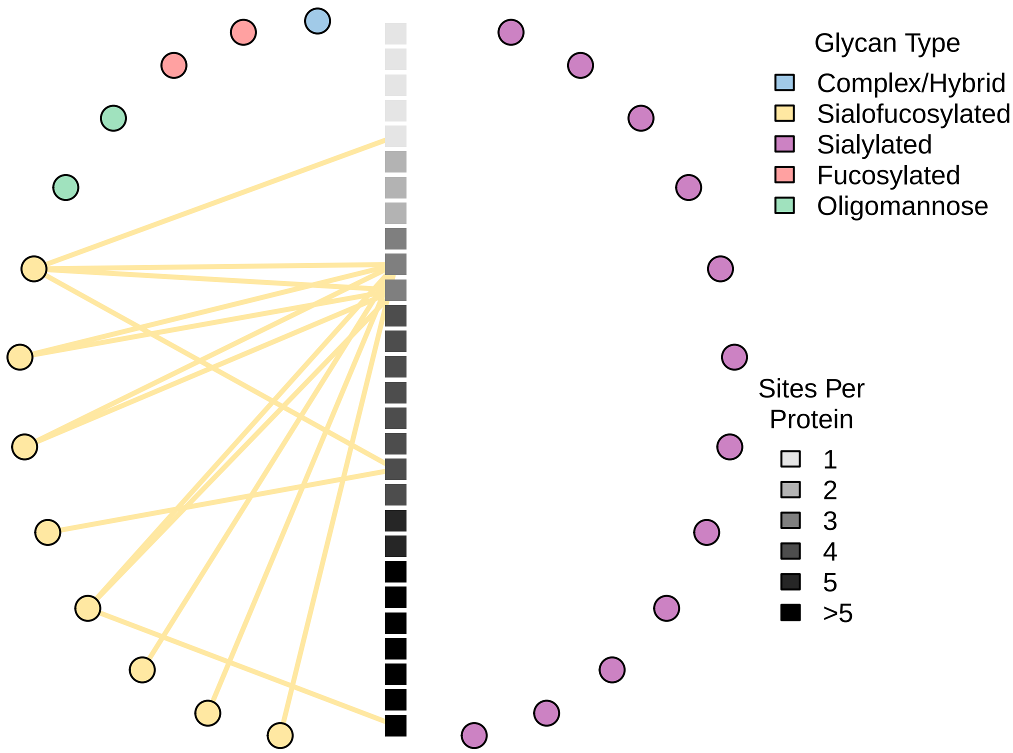
PlotGlycoProteinGlycanNetwork

A network graph where each node in the circle represents one glycan composition, and each node in the center represents one protein sorted by the number of identified N-glycan sites.

```
PlotGlycoProteinGlycanNetwork(exampleData)
```



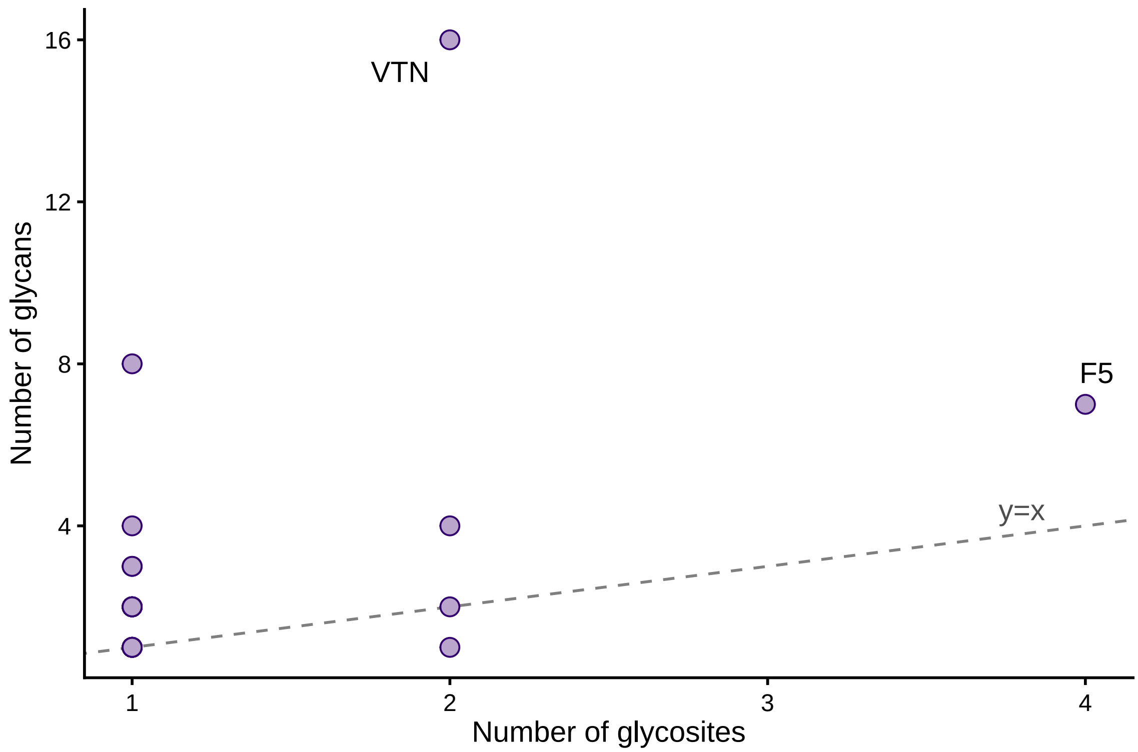

```
PlotGlycoProteinGlycanNetwork(exampleData, highlight = "Sialofucosylated",
                              verticeSize = c(6,7), edgeWidth = 2.5)
```



PlotGlycositesVsGlycans

The scatterplot where each dot is a protein, plotted by its number of glycosites (x-axis) versus the total number of glycans (y-axis).

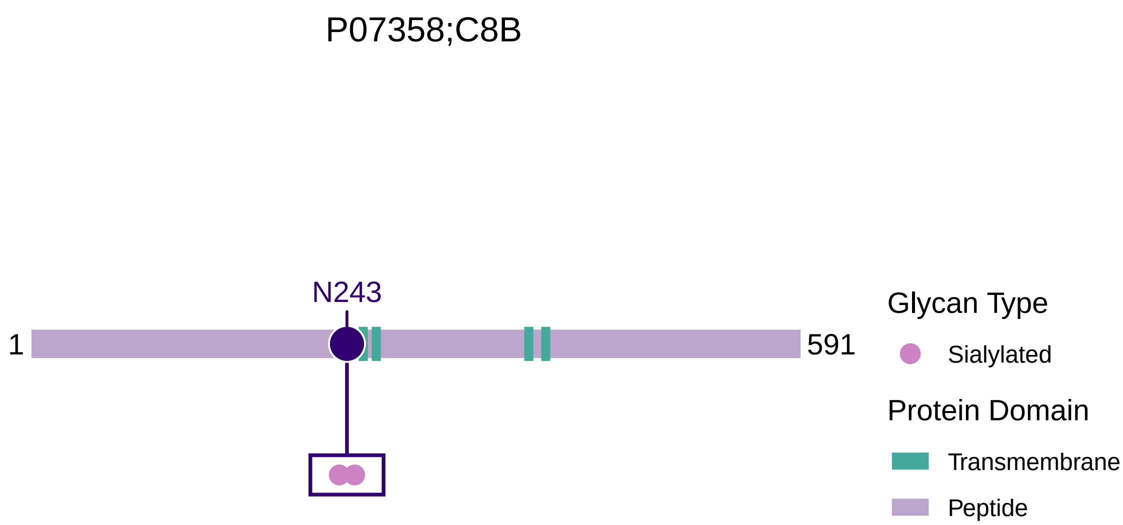
```
PlotGlycositesVsGlycans(exampleData, labelGlycositeCutoff = 1,
                        labelGlycanCutoff = 5,  alpha = 1)
```



PlotSiteGlycanComposition

A visualization per protein showing site heterogeneity of all glycosites. It automatically overlaps any domain information present on Uniprot.

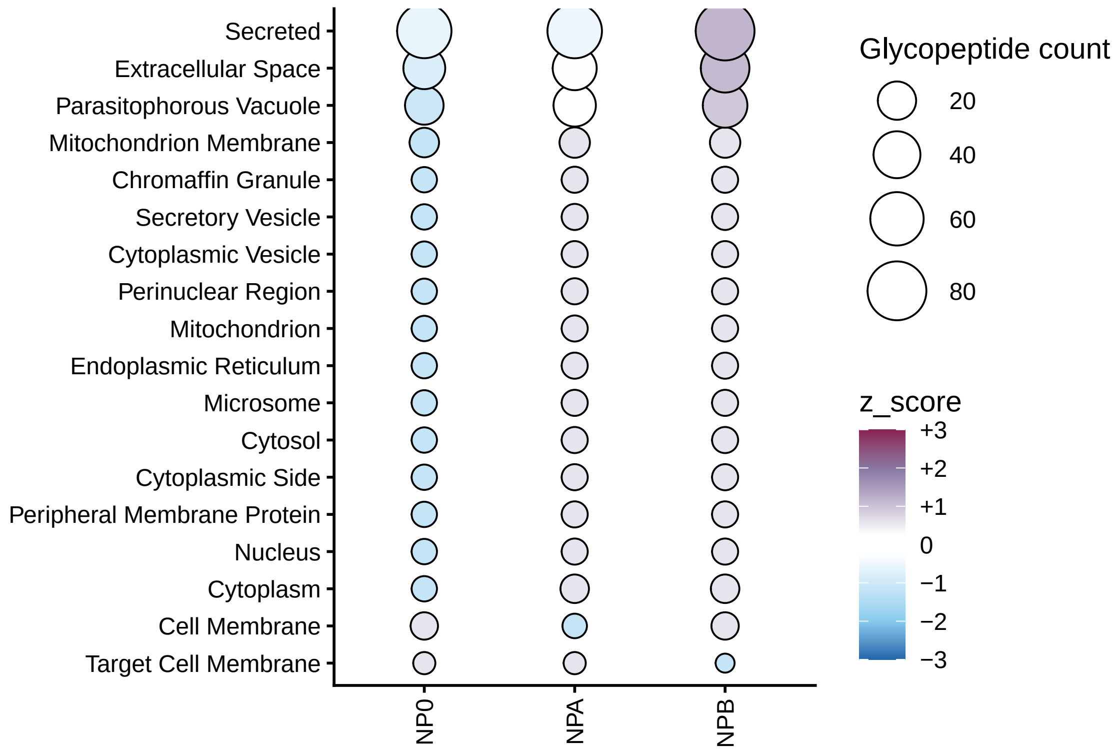
```
PlotSiteGlycanComposition(exampleData, whichProtein = "P07358")
```



PlotGlycanSubcellularLocation

Visualize the Uniprot subcellular location where each dot size is scaled by the number of unique glycopeptides found, and the color gradient is the Z-score of that subcellular location.

```r
PlotGlycanSubcellularLocation(exampleData, zscoreCutoff = 1)
```



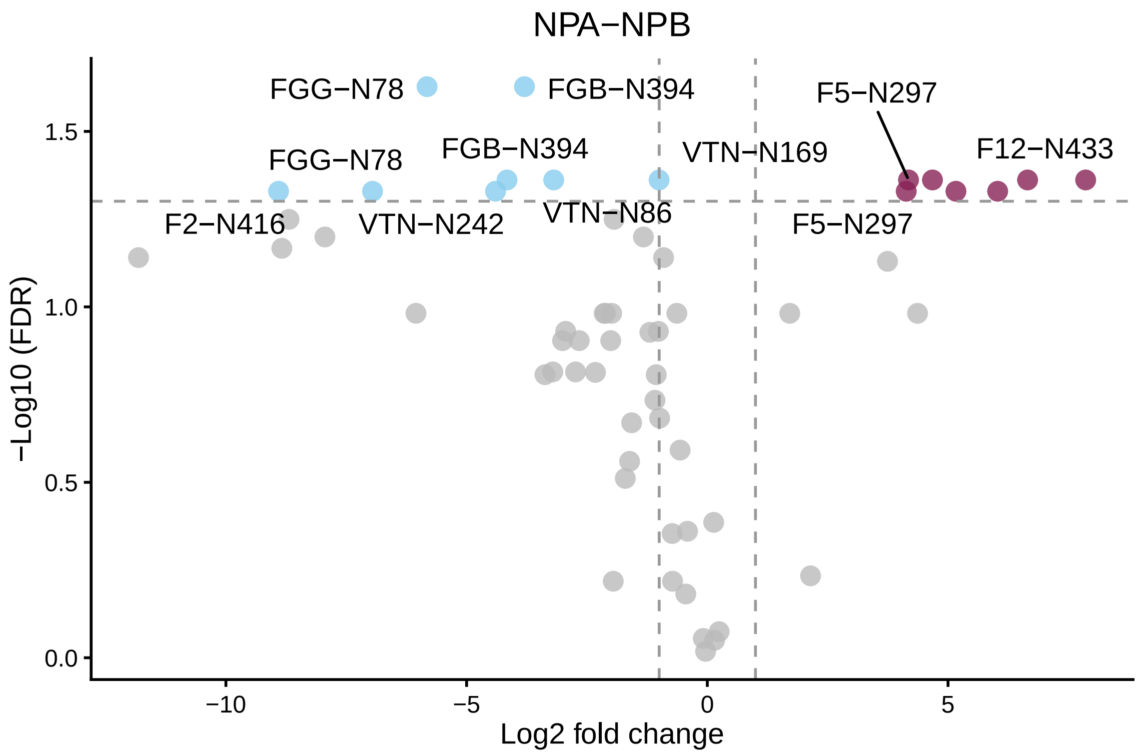
GlycoDiveR comparison plots

This needs comparison data. Please import MSstats or Perseus data. Or generate your comparisons like this. See the "Importing comparison results" section to see how.

PlotSiteGlycanComposition

A volcano plot.

```r
PlotComparisonVolcano(myComparison, whichComparison = c("NPA-NPB"),
                      statistic = "adjpvalue", statisticalCutoff = 0.05)
```



GlycoDiveR computation functions

GlycoDiveR has in-built functions to compute different types of data statistics.

ComputeMissingess

This function calculates the percentage of missing quantitative values on the glycopeptide level, either for all peptides, or for glycopeptides only. This will consider onsider peptides with an intensity of 0 as a missing quantitative value.

```
ComputeMissingness(exampleData, type = "all")
#> [1] "Percentage of missing values: 31.0833333333333%"

ComputeMissingness(exampleData, type = "glyco")
#> [1] "Percentage of missing values: 31.0833333333333%"
```

ComputeNumberOfGlycoforms

This computes the potential number of glycoforms based on the identified glycans. It will also include the unmodified site for each glycosite.

```
ComputeNumberOfGlycoforms(exampleData, whichProtein = "P00734,C9JV37,E9PIT3")
#> [1] 15
```

Saving plots

Generated plots can be treated as any other plot, and saved in all the common formats (pdf, png, svg, etc.). The first option is after running a plotting function:

```
ggsave("C:/filename.pdf", width = 7, height = 7)
```

The other option is to stream the plots to one, or more pdf files.

```
pdf("C:/filename.pdf", width = 7, height = 7)

Plotfunction1
Plotfunction2
Plotfunction3

dev.off()
```